

앞 글에서 depthwise가 라인버퍼로 3×3 이웃을 모은다는 것까지 정리했다. 이 글에서는 그 뒤단, **곱셈-누적을 DSP48로 어떻게 구성했고 왜 systolic 구조가 되었는지**를 핵심만 짚는다. 결과 구조는 **DSP 3체인 × 3탭 systolic**이고, 입력에서 출력까지 latency는 7이다.

한 줄 요약

fabric에 가산 트리를 만들지 않고, DSP 내부의 곱셈기·누산기·전용 캐스케이드 배선(PCOUT→PCIN , ACOUT→A)을 그대로 써서 3×3을 흘려보내며 누적한다.

왜 fabric 로직이 아니라 DSP48인가

3×3 depthwise 한 픽셀은 9번 곱하고 8번 더한다. 이걸 일반 fabric(LUT)로 만들면 48bit 가산기 8개 ≈ 수백 LUT가 든다. 그런데 FPGA의 **DSP48 슬라이스**는 곱셈기 + 48bit 누산기 + 파이프라인 레지스터 + **옆 DSP로 가는 전용 배선**을 이미 하드매크로로 갖고 있다.

- 곱셈을 DSP에 넣으면 fabric 가산 트리가 통째로 사라진다.
- OOC 합성 결과 **DSP 9 / LUT 41**. LUT가 41밖에 안 되는 것이 "가산을 fabric이 아니라 DSP 내부에서 했다"는 직접 증거다.

→ 곱셈-누적은 **DSP가 가장 잘하는 일**이라, 9개를 DSP에 얹고 부분합을 DSP끼리 직접 넘기도록 설계했다.

DSP를 두 종류로 나눈 이유 — head는 $A*B$, 나머지는 $A*B+PCIN$

DSP48 Macro IP를 두 개 만들었다.

IP	명령어	역할
dsp48_mul	$A*B$	체인의 맨 앞(head)
dsp48_mac	$A*B+PCIN$	체인의 2번째부터

굳이 나눈 이유가 있다. 부분합이 들어오는 PCIN 은 물리적으로 옆 DSP의 PCOUT 에서만 배선이 들어온다. fabric에서 상수 0을 물릴 수 없다. 따라서 **체인 맨 앞은 더할 부분합이 없으니 PCIN 을 아예 안 쓰는 $A*B$ 여야 한다.**

부수 효과

head를 $A*B+C$ (C는 fabric 포트)로 만들어 두면 나중에 BN bias 가산기가 공짜로 생긴다. 지금은 $A*B$ 로 두었다.

PCOUT → PCIN 캐스케이드 — 부분합이 1클럭에 한 칸씩

DSP 사이 부분합은 PCOUT → PCIN 전용 배선으로 넘어간다. 이걸 fabric을 안 거치는 **타일 내부 전용 경로**라 빠르고 라우팅이 깔끔하다. (같은 48bit라도 P는 fabric 어디로든 나가는 일반 출력, PCOUT 은 옆 DSP로만 가는 전용 출력이다. 그래서 인스턴스마다 둘 중 하나만 쓴다 — 중간 DSP는 PCOUT , 마지막 DSP만 P 로 결과를 꺼낸다.)

중요한 성질: 각 DSP의 출력 레지스터(PREG) 때문에 **부분합이 한 칸 내려가는 데 정확히 1클럭**이 걸린다. 이 1클럭/단이 systolic 타이밍의 뼈대가 된다.

Systolic이 하는 일 — 왜 9단 체인이 아니라 3체인 × 3탭인가

※ 처음 발상: 9개를 한 줄로 잇기 → skew 문제

DSP 9개를 한 줄로 캐스케이드하면, 부분합이 끝까지 내려가는 8클럭 동안 입력이 계속 흐른다. 같은 사이클에 9개 DSP에 값을 넣으면:

$$\text{DSP8 최종합} = a_8(t) + a_7(t-1) + a_6(t-2) + \dots + a_0(t-8)$$

└ 9개가 전부 다른 시각의 픽셀 = 서로 다른 출력의 탭 ─┘

같은 출력에 속한 9탭이 각자 자기 차례에 도착하도록 **입력을 미리 어긋나게(skew)** 넣어야 맞는다. 그런데 이 skew가 0~8 이 아니라 ** 0/3/6 ** 이다.

같은 행 안 : 열이 1칸 다르면 시각도 1클럭 차 → 캐스케이드 1클럭/단과 정확히 일치 (공짜)
 행 사이 : top/mid/bot 은 라인버퍼에서 같은 시각에 나오 → DSP 3칸 건너뛰니 3클럭씩 어긋남

DSP : 0 1 2 | 3 4 5 | 6 7 8
 행 : — top ———|—— mid ———|—— bot ———
 지연: 0 0 0 | 3 3 3 | 6 6 6

→ 열 방향 shift는 캐스케이드가 공짜로 해주고, 행 방향(3씩)만 손으로 맞추면 된다는 게 핵심 이해였다.

※ 최종 구조: 3체인 × 3탭 systolic (채택)

그런데 세 행(top/mid/bot)은 라인버퍼에서 **같은 시각에** 나온다. 그래서 한 줄 9단 대신 **행마다 독립된 3탭 체인 3개**로 나누고, 세 체인의 결과를 마지막에 더하는 구조로 바꿨다.

```

top_in → mac_top1 → ACOUT → mac_top2 → ACOUT → mac_top3 → top_out
           |          |          |
           PCOUT →    PCOUT →
mid_in → (동일 3탭 체인) → mid_out —————→ [+reg] → data_out
bot_in → (동일 3탭 체인) → bot_out —————→
  
```

실제 코드에서 체인 내부 weight는 **역순(k2 → k1 → k0)** 으로 물린다. 데이터가 캐스케이드를 타고 내려가며 연속한 세 열 샘플과 만나기 때문에, 각 샘플이 자기 커널 열(w[2]→w[1]→w[0])과 짝을 이루도록 순서를 뒤집는다.

```

dsp48_mul mac_top1(.A(top_in),      .B(weight[32+:16]), .ACOUT(a_top_in[1]), .PCOUT(pc_top[0])); // k2
dsp48_mac mac_top2(.A(a_top_in[1]), .B(weight[16+:16]), .ACOUT(a_top_in[2]), .PCOUT(pc_top[1]), .PCIN(pc_top[0])); // k1
dsp48_mac mac_top3(.A(a_top_in[2]), .B(weight[ 0+:16]), .P(top_out),      .PCIN(pc_top[1])); // k0
  
```

바꾼 이유: 세 행을 같은 시각에 소비하므로 0/3/6 skew 지연선(144 FF)이 통째로 사라지고, latency도 L_{AP}+8 → L_{AP}+3 (=7)으로 줄었다. 대신 48bit 3입력 가산기(~100 LUT) 하나가 생기는데, 이걸 충분히 이득이다.

✍️ 덤으로 사라진 것

이 구조에선 w[0:8] window 레지스터도 필요 없다. 캐스케이드의 지연이 열 방향 정렬을 대신하므로, 별도로 3×3을 레지스터에 쌓아둘 이 유가 없어졌다.

ACOUT로 A를 넘긴다 — ACIN 이 없어서

부분합은 PCOUT→PCIN 으로 넘기는데, **입력 A**도 다음 DSP로 넘겨야 systolic이 된다. 원래는 ACOUT→ACIN 전용 배선을 쓰고 싶었지만, DSP48 Macro IP에는 **ACIN 포트가 없다**(C_HAS_ACIN 은 켤 수 없는 내부 파생값이고, 전용 A 배선은 DSP48E1 primitive로만 가능).

그래서 차선책으로 **ACOUT 을 다음 인스턴스의 일반 A 입력에 연결**했다. fabric을 거치므로 전용 배선의 이점은 없지만 **데이터플로우와 타이밍은 systolic 그대로**다. 한 가지 성질만 기억하면 된다 — ACOUT 은 A를 **2클럭** 지연시킨 값이다(areg_3 + areg_4). 이 2클럭이 뒤에서 latency 계산

에 들어간다.

Latency 7의 구성

입력 input_data 부터 data_out 까지 정확히 7클럭이다.

input_data → [IP 내부 A→P : 4] → [PCIN 흡 x2 : 2] → [최종 가산 레지스터 : 1] → data_out
4 + 2 + 1 = 7

구간	클럭	근거
A → P	4	areg×2 + MREG + PREG
PCIN 흡 2번	2	3탭 체인이라 부분합이 2번 내려감
최종 가산 레지스터	1	세 행 합 top+mid+bot 을 1단 등록

DSP 구성을 바꾸면 이 숫자만 바뀌므로, testbench도 localparam LAT 으로 파라미터화해 두었다. areg 를 줄이면 latency는 줄지만 ACOUT 지연이 2→1이 되어 결과가 틀리므로 건드리면 안 된다.

파형으로 확인

weight를 전부 1로 두고 입력에 $x(t)=t$ 를 흘려 캐스케이드가 실제로 연속 세 샘플을 더하는지 확인했다.

[mac_top2] PCOUT = $x(t-6)+x(t-5)$ t=7 : 1+2 = 3 ✓ (1*1+1 이 아님)
[mac_top3] P = top_out = $x(t-8)+x(t-7)+x(t-6)$ t=7 : 0+0+1 = 1 ✓

top_out 이 연속한 세 샘플의 합이라는 것은 3×3의 한 행이 정상 누산됐다는 뜻이다. 동시에 A: 1 2 3 ... 대비 ACOUT: _ _ 1 2 ... 로 ACOUT 2클럭 지연도 값으로 확인됐다. 전체 시뮬레이션은 LAT=7 에서 checked 392/392, errors 0으로 통과했다.

나의 질문

Q. PCOUT 이 왜 1*1+1=2 가 아니라 2*1+1=3 으로 나오는가?

A. A와 PCIN이 서로 다른 파이프라인 깊이에서 도착하기 때문이다. A는 areg×2 → MREG → PREG 로 곱셈기에 도달할 때 이미 2클럭 지난 값 ($x(t-2)$) 인데, PCIN은 PREG 만 거쳐 1클럭 만에 들어온다. 그래서 그 순간 곱해지는 A는 1 이 아니라 이미 2 가 된 값이라 합이 3이 된다. 파이프라인 각 포트의 도착 시각을 따로 세지 않으면 틀리기 쉬운 부분이다.

결론 : 곱셈-누적을 fabric 대신 DSP48에 얹고, 부분합은 PCOUT→PCIN 전용 배선으로, 입력은 ACOUT→A (2클럭 지연)로 넘기는 systolic 구조로 만들었다. 세 행이 라인버퍼에서 동시에 나오는 점을 살려 9단 단일 체인(skew 0/3/6 필요) 대신 3체인×3탭으로 구성해 skew 지연선과 window 레지스터를 없앴고, latency 7로 checked 392/392 errors 0을 통과했다.